

Amortized Custom eFPGA Layout Search via Reinforcement Learning: Zero-Shot Generalization Across Circuit Benchmarks

Anonymous Author(s)

Abstract

Custom eFPGA fabrics embed reconfigurability directly in fixed silicon — enabling domain-specific acceleration, post-fabrication protocol updates, and IP security on a single chip — but every search-based layout method to date restarts from scratch for each new netlist, repeating hundreds of expensive VTR evaluations with no information transferred from prior runs. We train a reinforcement-learning policy, conditioned on a graph-neural-network encoding of the target netlist, to choose DSP/BRAM placement and fabric aspect ratio across a pool of 11 circuits; the same checkpoint then applies zero-shot to circuits never seen in training. Under a controlled comparison against a genetic algorithm sharing the same oracle, metric, and baseline, a zero-shot rollout delivers an immediate positive result in a single oracle call while the GA requires 768 VTR evaluations and ~ 6.6 hours — 50 of those generations spent stagnating after its best was already found. Trained jointly, the policy achieves 27.35% aggregate in-pool ADP reduction (three seeds); applied zero-shot to six held-out circuits — including fabrics up to $3.4\times$ the training area — every result is positive, the average is 31.23%, and seed variance is markedly lower than in-pool fitting.

1 Introduction

Embedding a small reconfigurable fabric in an ASIC or SoC — an eFPGA — lets a fixed design adapt after fabrication: domain-specific acceleration, post-fabrication protocol updates, IP security, and extended product life can all be managed through the bitstream rather than a new chip revision [20, 21]. A generic FPGA is designed for arbitrary workloads and is therefore over-provisioned for any single application — extra routing, DSP columns, and BRAM arrays that the deployed netlist never fully uses. A custom eFPGA fabric, sized and arranged for one specific netlist rather than general-purpose reconfigurability, can recover much of that overhead. The product of routing area, critical-path delay, and dynamic power — which we abbreviate **ADP** throughout — quantifies this overhead directly: it measures how much the eFPGA fabric costs relative to hardening the same function in standard cells, and minimizing it benefits every eFPGA application [3, 20]. Note that GOLDS [20] uses the same abbreviation for the two-term $\text{Area}\times\text{Delay}$ product; in Section 5.3 we recompute results on their exact two-term metric to make the comparison direct.

Two prior papers establish that per-instance search over DSP/BRAM placement (and, in one case, CLB/arithmetic-block sizing) can substantially reduce the eFPGA fabric overhead: GOLDS [20] uses a genetic algorithm with an $\text{Area}\times\text{Delay}$ fitness function and reports up to $\sim 35\%$ $\text{Area}\times\text{Delay}$ improvement on the *diffeq1* benchmark; a follow-up applies NSGA-II to a larger, multi-objective search space that additionally co-evolves CLB LUT size and custom arithmetic-block sizing [3]. Both, however, are *testbench-specific*: the genetic algorithm’s population is re-initialized and re-evolved from scratch for every new target design, and every generation of every run

requires its own full, expensive VTR evaluation. Nothing learned while optimizing one netlist’s layout carries over to the next netlist — each new target benchmark pays the full search cost again, with no way to amortize the work already done on previous designs.

We train a single RL policy, conditioned on a GNN encoding of the target netlist, to choose DSP/BRAM placement and fabric aspect ratio for a custom eFPGA fabric — the problem where per-instance GA search has been the only option. Unlike RL methods that assign blocks to an already-fabricated grid, our policy *constructs* the custom fabric layout itself. Trained jointly across 11 circuits and scored by the real VTR flow at every step, the same checkpoint then applies zero-shot to circuits never seen in training, including fabrics up to $3.4\times$ the area of anything in the training pool. Training runs once; each new netlist costs one rollout. We quantify the search-cost gap against a from-scratch genetic algorithm under an identical oracle, metric, and baseline: zero-shot, the policy delivers an immediate positive result in one call; fine-tuned, it matches the GA’s final quality while the GA spends the majority of its 768 evaluations stagnating after its best was found early (Section 5.2).

Concretely, this paper contributes:

- A GNN-conditioned RL policy for custom eFPGA DSP/BRAM placement and aspect-ratio selection. A fixed-size network encodes each target netlist so the same checkpoint applies to circuits never seen during training, evaluated directly against the VTR flow rather than a proxy (Sections 3.3, 3.4).
- A controlled cost comparison against a from-scratch genetic algorithm under an identical evaluation oracle, ADP metric, and baseline, on the held-out *sof tmax* benchmark: zero-shot, the policy delivers an immediate positive result in one oracle call; fine-tuned, it matches the GA’s final ADP reduction while the GA expends 768 evaluations (~ 6.6 h) — stagnating for the final 50 generations after its best individual was found at generation 46 (Section 5.2).
- A joint 11-benchmark training result (27.35% aggregate in-pool ADP reduction, averaged over three seeds) (Section 5.1), and a literature-facing comparison against GOLDS’ published metric and baseline, recomputed on a zero-shot evaluation: 37.4% against GOLDS’ $\sim 35\%$ (Section 5.3).
- A three-seed reliability analysis exposing an asymmetry: in-pool fitting is far more seed-sensitive (two benchmarks spanning ~ 28 points across seeds) than zero-shot generalization (at most ~ 12 points across all six held-out benchmarks), reported as raw per-seed results rather than mean $\pm$ SD (Sections 5.4, 5.6).

2 Background and Related Work

2.1 Custom eFPGA Layout Design

A generic island-style FPGA provisions DSP columns, BRAM columns, and routing for arbitrary workloads; any single application leaves much of this capacity unused, paying area and delay overhead for

resources it never exercises. This has motivated a line of work on *custom* eFPGA layout: tailoring tile placement and fabric dimensions to a specific target netlist so the deployed fabric is compact and efficient rather than over-provisioned. Applications driving this work include IP security (where the function implemented on the eFPGA is absent from fixed silicon until configured [6, 21]), domain-specific SoC acceleration, and post-fabrication reconfigurability [20]. Subsequent work reduces eFPGA overhead further by shrinking under-utilized routing [15] or de-regularizing the fabric itself [8] — both motivating the custom, non-island-style layouts this paper synthesizes. All rely on heuristic or evolutionary search re-run per design; none amortizes layout effort across netlists. GOLDS [20] introduced custom-tailored eFPGA layout search: a genetic algorithm over DSP/BRAM tile placement on a fixed-size fabric, with an Area×Delay fitness evaluated through the real VTR flow, reporting up to ~35% Area×Delay improvement over a traditional island-style layout on diffeq1. A follow-up [3] extends this to a multi-objective NSGA-II search that additionally co-evolves CLB LUT size and custom DSP/BRAM block sizing — not just placement — optimizing delay and power jointly and reporting up to ~44.6% improvement on a larger design space. Both are strong results; both start from a random population for every new target design, transferring no information from prior runs. We target the same problem GOLDS defines and use the same VTR oracle, which gives us a clean controlled comparison (Section 5.2); the NSGA-II co-evolution of block sizing is a natural extension for future work (Section 6).

2.2 Learned Policies for Physical Design

Reinforcement learning has been applied to chip-scale physical design. AlphaChip [11, 18] trains an RL agent to place ASIC macro blocks using a learned reward proxy, and reports policies that transfer across chip designs without retraining — the amortization argument we make here, at a much larger design scale. Subsequent work explores visual state representations [17], joint placement-and-routing [5], and offline RL over a transformer policy [16]. All motivate the claim this paper tests in a smaller-scale setting: a trained policy amortizes layout search cost across designs in a way that per-instance search cannot. Unlike AlphaChip, we evaluate every candidate against the true VTR flow rather than a learned proxy, trading speed for fidelity — affordable at our scale, where a VTR run completes in seconds to low minutes (Section 4.1).

Within the FPGA setting specifically, RL has guided placement onto an *already-fabricated* grid — selecting annealing perturbations in VPR [10] or driving a Gym-style VPR environment [4] — but always assigns blocks to a fixed grid, never shapes the grid itself. Transferable EDA policies have appeared for tool tuning [13] and global placement [12]. Our policy’s role is different: it synthesizes the custom FPGA fabric geometry rather than placing onto an existing one.

2.3 VTR/VPR Background

We build on the open-source Verilog-to-Routing (VTR) [2, 9, 19] toolchain: Yosys performs synthesis (via the VTR integration [7]) and VPR performs packing, placement, and routing against an architecture described in an XML schema of tile types, positions,

and connectivity. This architecture XML is the artifact GOLDS and the NSGA-II follow-up generate by hand-coded search and that we generate by policy rollout (Section 4.1); VTR’s acceptance of an arbitrary heterogeneous tile layout, not only the regular island-style grids it ships, is what makes this entire line of work possible.

3 Methodology

3.1 Problem Formulation

A clarification on scope. The reconfigurability in this setting is not general-purpose: a fabric optimized for one netlist will not accommodate a circuit with substantially different DSP/BRAM requirements. This is intentional — the tradeoff is reduced cross-circuit generality in exchange for a significantly smaller, more efficient implementation. Reconfigurability means updating the same primary function (parameter changes, post-fabrication fixes, a known set of operational modes), not repurposing the fabric for an unrelated workload. Optimizing for one target netlist is the design goal.

We separate two roles that are easily conflated. The *policy* designs the fabric; the *VTR toolchain* maps the circuit onto it. Concretely, given a netlist requiring n_{dsp} DSP and n_{bram} BRAM hard blocks, the policy emits an architecture descriptor $\pi = (\text{ar}, \{(x_i, y_i)\}_{i=1}^{n_{\text{dsp}}+n_{\text{bram}}})$: a fabric aspect ratio $\text{ar} \in \mathcal{R}$ and a tile coordinate for every hard block. VTR then runs the full synthesis-to-routing flow on that architecture — it synthesizes the Verilog, packs the remaining logic into CLB clusters, places every block (CLBs included), routes, and auto-sizes the array to the smallest grid of the chosen aspect ratio that holds the design. The policy thus never places a CLB or fixes the grid dimensions: it shapes the fabric and positions the hard blocks, and VTR reports back the realized cost. We take that cost to be the product of routing area, critical-path delay, and dynamic power (our three-term ADP metric, as defined in Section 1):

$$\text{ADP}(\pi) = \text{Area}(\pi) \times \text{Delay}(\pi) \times \text{Power}(\pi), \quad (1)$$

where each term is reported by the VTR (Yosys + VPR) flow (Section 4). We use VPR’s *routing area* — the physical tile-grid footprint including routing channels — rather than a logical area measure such as CLB count. The policy controls fabric geometry (aspect ratio, tile positions), which changes routing area; logical CLB area is determined by the netlist and is approximately layout-independent. Routing area is therefore the metric that responds to policy decisions and reflects the actual on-chip die area of the deployed eFPGA. We report results as percentage ADP reduction relative to a traditional, non-customized island-style eFPGA baseline at the same logic capacity.

3.2 RL Formulation

We pose placement as a finite-horizon sequential decision process and train a policy with Proximal Policy Optimization (PPO), using the maskable variant [14] (sb3_contrib’s MaskablePPO) so that the action space can be restricted to currently-legal tile coordinates at every step.

Episode structure. An episode lays out exactly one benchmark’s hard blocks on the shared canvas. Step 0 selects the fabric aspect ratio from a fixed discrete set $\mathcal{R}$; each subsequent step positions one hard block — BRAMs first, then DSPs, each group ordered by descending shared-net connectivity, so the most-connected blocks

are committed earliest. Once all hard blocks are positioned, the environment bakes the layout into an architecture XML and a placement-constraints file (via Jinja2 templating) and invokes the VTR flow to obtain $\text{Area}(\pi)$, $\text{Delay}(\pi)$, and $\text{Power}(\pi)$; all intermediate steps receive zero reward.

Action space. The action space is $\text{Discrete}(W_{\max} \times H_{\max})$, fixed across all benchmarks in a training run (Section 3.4). At step 0 an action index selects an entry in $\mathcal{R}$; at later steps an action index decodes to a tile coordinate (x, y) via a fixed stride on $H_{\max}$, independent of which benchmark is active. Action masking — keyed off the *active* benchmark’s real width/height and the blocks already placed — restricts the policy to legal, non-overlapping, in-bounds coordinates for that specific benchmark and step.

Reward. The terminal reward is a weighted sum of negative log-ratios against the traditional baseline for the active benchmark:

$$r = -\left(w_a \log \frac{\text{Area}}{\text{Area}_0} + w_d \log \frac{\text{Delay}}{\text{Delay}_0} + w_p \log \frac{\text{Power}}{\text{Power}_0}\right), \quad (2)$$

where $\text{Area}_0, \text{Delay}_0, \text{Power}_0$ are the traditional-baseline metrics for that benchmark and w_a, w_d, w_p are configurable weights (Section 3.5). With $w_a = w_d = w_p = 1$, r equals $\log(\text{ADP}_0/\text{ADP})$ — i.e. the reward is the log of the ADP-improvement ratio in Eq. 1, making reward and our headline metric directly comparable. An invalid choice (degenerate aspect ratio, out-of-bounds or overlapping placement, or a VTR flow failure) terminates the episode early with a fixed penalty reward of -10 .

3.3 GCN Feature Extractor

To let a single policy generalize across benchmarks with different netlist sizes, hard-block counts, and grid dimensions, we encode each netlist as a graph rather than a flat vector, giving the policy the inductive bias to transfer across circuit topologies that graph learning provides elsewhere in EDA [22]. We first reduce each benchmark’s full netlist graph to a compact, fixed-schema representation: one node per DSP/BRAM hard-block instance, plus one aggregate “fabric” node summarizing the remaining standard-cell logic. Each node carries a 4-dimensional feature vector $[\text{is_dsp}, \text{is_bram}, \text{is_fabric}, \text{net_count_normalized}]$. Edges between two hard-block nodes are weighted by their raw shared-net count; edges between a hard-block node and the fabric node are weighted by the summed shared-net count across the standard cells that block touches. As these two edge kinds live on very different scales, we normalize each into $[0, 1]$ separately by a log compression $w_{\text{norm}} = \log(1 + w)/\log(1 + c)$ with a per-kind ceiling c from the training pool, so neither type saturates or vanishes.

This reduced graph is padded to a fixed $(\text{MAX_NODES}, \text{MAX_EDGES})$ — computed once over the full benchmark *universe* (Section 3.4) — and fed to a small GNN feature extractor shared by the policy and value heads: two GCNConv layers (hidden width 64) produce per-node embeddings h_i ; a global mean pool gives a whole-netlist embedding, and the embedding of the node being placed this step gives a focused per-step embedding. These are concatenated with a small CNN over the occupancy grid (4 channels: occupancy, block-type hint, aspect ratio, net count) and the active benchmark’s normalized (width, height), then projected to a 128-dimensional vector for the PPO policy and value heads. The extractor is trained end-to-end under the PPO loss, with no separate GNN pretraining.

3.4 Universe Superset and Action Masking

A policy’s input/output dimensions are fixed at construction, yet we want one checkpoint loadable against benchmarks never trained on, including ones discovered later. We therefore size every dimension-dependent quantity — canvas $(W_{\max}, H_{\max})$, MAX_NODES , MAX_EDGES — from a benchmark *universe* that is a deliberate superset of the sampled training set, including benchmarks reserved purely for zero-shot evaluation, provided none exceeds the shared dimensions. Per-episode action masking (Section 3.2) and the `valid_wh` observation let the same fixed-size network operate on each benchmark’s smaller real footprint within that canvas.

3.5 Training Setup

Each training run samples uniformly across the 11-benchmark training pool (Figure 1) episode-by-episode, so a single policy is updated jointly on all 11 circuits rather than one at a time. We use MaskablePPO with 24 parallel environments, 48 rollout steps per environment per update, batch size 144, 15 epochs per update, entropy coefficient 0.01, and reward weights $w_a = w_d = w_p = 1$ (Eq. 2), giving reward = $\log(\text{ADP}_0/\text{ADP})$. Each run trains for 13,000 episodes (capped independently of total timesteps) with a per-episode VTR timeout of 300s to keep a degenerate early-exploration episode from stalling the other 23 parallel workers. We report results from three independent training seeds (7, 42, 123) under this identical configuration, used throughout Section 5 to characterize result reliability rather than report a single run’s numbers as ground truth.

4 Experimental Setup

4.1 Evaluation Oracle

Every candidate placement is scored by the true downstream toolchain, not a proxy. A placement (aspect ratio + hard-block coordinates) is rendered into a complete VTR architecture XML and a VPR placement-constraints file via Jinja2 templates parameterized by tile coordinates, block type, and fabric aspect ratio. This architecture and the benchmark’s Verilog are then run through the full VTR flow — Yosys-based synthesis, VPR packing, placement, and routing, followed by VPR’s power estimator against a 45nm PTM technology file — and we extract routing area, critical-path delay, and dynamic power from the resulting VPR reports. All evaluations use a VTR 9.0.0 development build (git 22a09d39, GCC 13.3.0). Evaluated (placement, benchmark) pairs are cached, since the synthesis-to-routing flow is the dominant cost of both training and evaluation (seconds to low minutes per call depending on benchmark size).

4.2 Baseline

Our traditional baseline is VTR’s default island-style architecture template (k6_frac_N10_mem32K_40nm: 6-input LUTs, cluster size 10, 32K-bit BRAMs, 40nm) at matched logic capacity for each benchmark — the same template used by GOLDS [20] as its traditional-layout comparison point, which lets us compare percentage improvements directly against theirs on a shared starting point (Section 5.3).

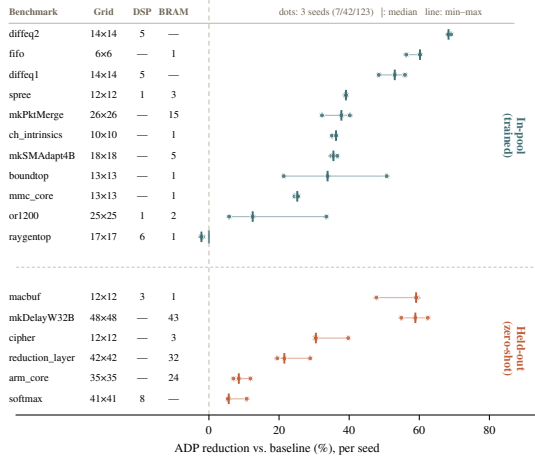


Figure 1: Per-seed ADP reduction for all 17 benchmarks; row labels carry the benchmark pool metadata (grid size and hard-block counts: D=DSP, B=BRAM) in place of a separate table. Dots are the three training seeds (7/42/123), the tick is the median, and the line spans min-max. In-pool fitting is seed-sensitive on or1200 and boundtop (shaded), spanning ~28–29 points; every other benchmark stays within ≤ 12 points. The held-out zero-shot group is more seed-stable than in-pool fitting.

4.3 Benchmarks

Figure 1 lists all 17 benchmarks with their grid size and hard-block counts. The pool comprises an 11-benchmark training set of real circuits spanning a range of DSP/BRAM resource profiles, and a held-out set of six circuits used only for zero-shot evaluation (Section 5.4), never sampled during training. Training circuits come from the standard VTR suite [19]; three held-out benchmarks (softmax, reduction_layer, arm_core) together with mkDelayWorker32B are larger standard circuits (from Koios [1] and the VTR suite) that also probe fabric sizes well beyond the training pool. Because the standard suites contain few *small* heterogeneous designs, we add two small probes — lightweight_cipher and custom_macbuf, the latter an internally built multiply-accumulate buffer — to retain coverage at the small-fabric end of the held-out set. The GA head-to-head (Section 5.2) uses softmax, a standard Koios benchmark held out of RL training.

4.4 Baselines Compared

We compare against: (1) the traditional island-style eFPGA baseline (Section 4.2); (2) a genetic algorithm we implemented under the same evaluation oracle and ADP metric (Section 5.2); and (3) published results from GOLDS [20] and the NSGA-II follow-up [3], subject to the metric caveat in Section 5.3.

5 Results

5.1 In-Pool Performance

Figure 1 (upper group) reports per-benchmark in-pool ADP reduction for the 11-benchmark policy on its own training set across three

independent seeds (7, 42, 123; Section 3.5). We show all three per-seed outcomes rather than a mean $\pm$ SD, which three runs cannot justify (Section 5.6). Each seed’s pooled aggregate — one minus the ratio of summed ADP across all 11 benchmarks to summed baseline ADP, *not* a simple mean of per-benchmark percentages — is 35.73%, 23.11%, and 23.97% (mean 27.35%). raygentop is consistently negative across all three seeds ($[-2.2, -1.7]\%$). Its 6-DSP/1-BRAM profile is the only one in the pool combining a large DSP count with a BRAM (other DSP-heavy benchmarks have no BRAM; other BRAM-heavy ones have ≤ 1 DSP), and despite a fair $\approx 9.1\%$ episode share the policy finds no improving placement for this configuration — plausibly a representational gap rather than a convergence shortfall, and our most immediate diagnostic target. Two benchmarks, or1200 and boundtop, account for almost all of the seed-to-seed instability: their per-seed outcomes span 5.8–33.5% and 21.3–50.7% respectively, ranges three to four times wider than any other benchmark in the pool (all ≤ 8 points). We analyze this asymmetry in Section 5.6.

5.2 GA Head-to-Head Comparisons

To establish a controlled cost comparison — without the metric-conversion caveat of Section 5.3 — we implemented a GA under the same oracle, ADP metric, and baseline as the RL pipeline and ran it on softmax: roulette-wheel selection, single-point crossover, and per-gene mutation over hard-block coordinates and fabric aspect ratio, population 8 (matching the eight parallel RL workers), capped at 500 generations with patience 50.

Table 1 reports the outcome. The GA’s best individual — **14.62%** ADP reduction, comparable to the fine-tuned policy’s **14.82%** — appeared at generation 46, after 368 VTR evaluations and ~ 2.75 h of wall-clock. The GA then ran for 50 more generations (400 additional evaluations, ~ 3.8 h) without improvement before the patience condition terminated it, for a total of 768 calls and ~ 6.6 h. Zero-shot, the policy achieves **5.39%** ADP reduction in a *single* oracle call (~ 134 s); fine-tuned, it matches the GA’s quality at $\sim$ episode 552 (~ 4 h) and continues to 14.82% at episode 1,128 (~ 8 h).

The timing exposes a structural difference. The GA’s best appeared at generation 46 and never improved; the remaining 50 generations were spent on a stagnated population that could not escape the local optimum. Per-generation averages confirm this: the typical individual is worse than baseline in *every* generation — the GA carries forward a lucky draw but never updates its sampling distribution. Gradient updates do exactly this for the RL policy: the fraction of episodes beating baseline rises from $\sim 10\%$ early in fine-tuning to $\sim 75\%$ by the end, which is why the policy continues improving past the point where the GA stalled. The GA’s evaluation cost is re-incurred from scratch for every new target design; the policy’s training cost is fixed, so the amortization advantage grows with the number of designs a team must handle.

Figure 2 shows where the saving comes from physically on softmax. The traditional fabric provisions BRAM columns the netlist never uses, allocating area to a generic, over-provisioned array. The policy’s architecture omits the unused BRAM resource entirely and selects an aspect ratio under which VTR auto-sizes the same logic onto a 22% smaller array — the dominant term in the ADP reduction. The policy never moves a CLB; it removes idle

Table 1: GA vs. RL on softmax (held out of RL training; baseline ADP = 3.58×10^6). GA: pop. 8, ≤ 500 gens, patience 50; best at gen. 46, stagnated the final 50 gens. RL zero-shot: single deterministic rollout (seed 42). RL fine-tuned: 11-benchmark policy continued on softmax. Same 24-core machine throughout.

Method	Reduction	VTR calls	Wall-clock
GA	14.62%	768	~6.6 h
RL zero-shot	5.39%	1	~2 min
RL fine-tuned	14.82%	1,128	~8 h

resource columns and reshapes the fabric, and the tighter routing that follows is VTR’s.

5.3 Head-to-Head Against GOLDS

As a secondary, literature-facing comparison, we recompute a genuine zero-shot result — a policy trained only on `diffeq2`, evaluated zero-shot on `diffeq1` with no benchmark-specific training — using GOLDS’ [20] exact two-term $\text{Area} \times \text{Delay}$ metric and the same traditional baseline architecture they report against. Two differences from the rest of this paper apply: (1) we substitute the two-term $\text{Area} \times \text{Delay}$ product for our three-term $\text{Area} \times \text{Delay} \times \text{Power}$ metric, and (2) we note that GOLDS’ area term is a logical measure of fabric utilization while ours (Section 3.1) is VPR’s routing area; to match GOLDS as closely as possible we recompute on their metric and baseline rather than ours. This yields a **37.4%** reduction versus GOLDS’ published best-of-population result of $\sim 35\%$ on the same benchmark — a real, if narrower, margin, achieved with zero benchmark-specific training. Nowhere else in this paper do we compare two-term numbers against our three-term results directly.

5.4 Zero-Shot Generalization

Figure 1 (lower group) reports ADP reduction on all six held-out benchmarks (Figure 1), across the same three seeds as Section 5.1, evaluated with zero benchmark-specific training. Two of the six (`softmax`, `reduction_layer`) were part of the training-time *universe* used to size the policy’s tensors (Section 3.4) but never sampled during training; the other four were entirely outside the universe as well, a strictly harder generalization test. Crucially, four of the six are fabrics *larger than anything in the training pool* (up to 2,304 tiles versus the pool’s 676), so this is size extrapolation, not interpolation. Every benchmark finishes positive under every seed — no sign flips at all, unlike the in-pool result — and every per-seed range is narrow (at most ≈ 12 points, for `custom_macbuf`), with no benchmark approaching the `or1200/boundtop` spread. The overall average across all six benchmarks and three seeds is **31.23%**. The weakest benchmarks (`softmax`, `arm_core`) show proportionally larger seed spread relative to their mean, but the absolute range remains small (at most ≈ 12 points) in every case.

All reported numbers use greedy (deterministic) decoding, which is the intended deployment mode. As a diagnostic, stochastic rollouts on one seed yielded negative average ADP reductions on 4 of 6 benchmarks, confirming that the learned distribution is concentrated: the greedy action occupies a high-quality region while most

of the probability mass does not. The zero-shot results should be read as the top-choice performance per seed, not an expectation over sampling.

5.5 Fine-Tuning Efficiency

We also characterize continued learning on a partially-known benchmark (`diffeq1`, in the training pool): fine-tuning the multi-benchmark policy versus training from scratch, both 6,000 episodes under identical hyperparameters and separate caches. The fine-tuned run dominates typical reward throughout (0.785 vs. 0.736 in the final window) and plateaus by $\approx$ episode 3,000, while scratch never plateaus within its budget. The mechanism is entropy collapse: the fine-tuned policy’s entropy falls from 2.00 to 0.79 (vs. scratch’s 4.63 to 2.42), so only 31.5% of its episodes are novel VTR evaluations versus 94.4% for scratch — a sample-efficiency win that, as Section 5.6 notes, also caps further exploration beyond the pretrained prior. This is continued learning, not cross-benchmark transfer (Section 5.4).

5.6 Seed Variance: In-Pool vs. Zero-Shot

Across three independent training seeds, the 11-benchmark policy exhibits a clear asymmetry: *in-pool* performance is considerably more seed-sensitive than *zero-shot* performance. For nine of the eleven training benchmarks, the per-seed range is small (≤ 8 points); two benchmarks, `or1200` and `boundtop`, span ~ 28 and ~ 29 points respectively — three to four times the spread of any other benchmark — and account for almost the entire spread in the pooled aggregate (35.73% / 23.11% / 23.97% across seeds). On the six held-out zero-shot benchmarks, by contrast, every per-seed range stays within ~ 12 points (Figure 1); the two shaded in-pool benchmarks have the widest ranges in either group.

We do not have a confirmed mechanism but propose one. In-pool performance depends on which benchmarks a given seed converges on slowly within the shared episode budget, whereas zero-shot performance reflects a more general property of the learned representation (Section 3.3), independent of any one benchmark’s convergence. The entropy collapse of Section 5.5 compounds this: a seed converging early on most benchmarks retains fewer episodes for a slow-to-converge benchmark such as `or1200`. No seed leads on both high-variance benchmarks and elsewhere at once (seed 42 leads on `or1200` and `boundtop`; seeds 7 and 123 lead on others), pointing to per-benchmark convergence variance rather than a globally better seed. At $n = 3$ this is a characterization; more seeds would establish whether the asymmetry is stable.

6 Conclusion

A single policy trained across 11 circuits answers the cost question directly: under a controlled comparison on an identical oracle, ADP metric, and baseline, the GA stagnates after generation 46 and spends 768 total evaluations and ~ 6.6 h finding nothing beyond its early best. The fine-tuned policy matches that quality and keeps improving; zero-shot, the policy delivers an immediate positive result in one call (Section 5.2). The same policy generalizes zero-shot to six held-out benchmarks — four of them fabrics larger than anything seen in training — and, recomputed on GOLDS’ published metric, exceeds their reported result (Sections 5.3, 5.4).

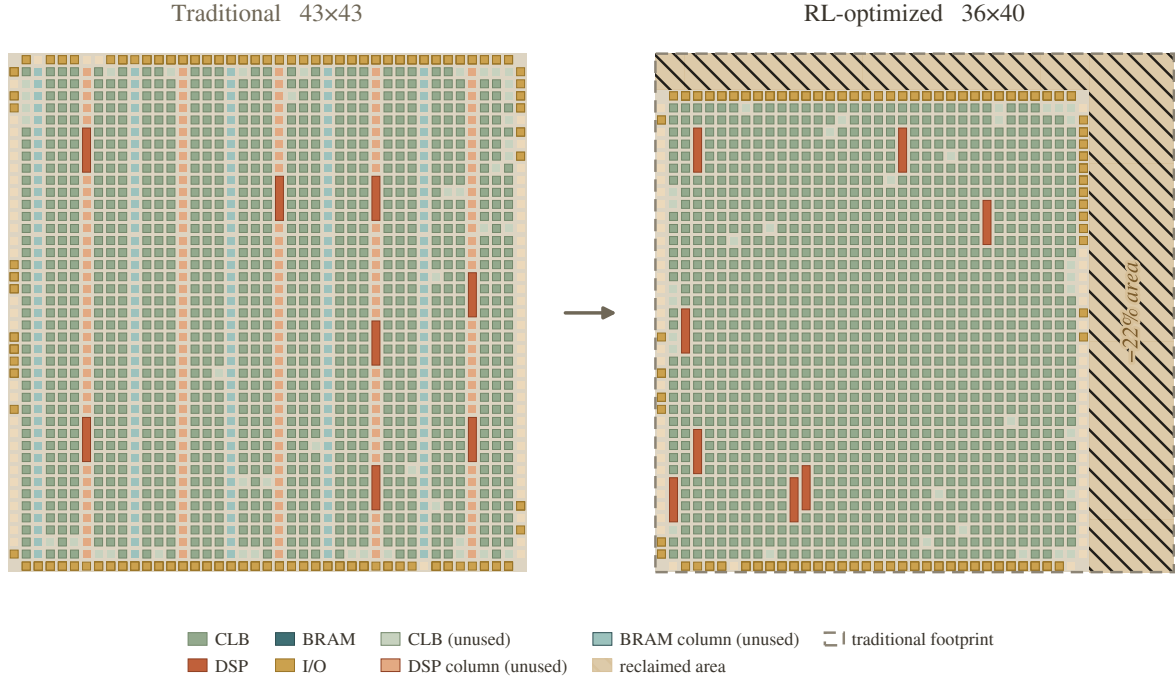


Figure 2: What the policy recovers, physically (softmax, the GA comparison benchmark). Both fabrics are drawn at the same tile scale. The traditional island-style baseline (left) is a 43×43 heterogeneous fabric that provisions BRAM columns (plum) the netlist never uses, with DSP blocks (terracotta) spread across regular DSP columns. The RL-optimized fabric (right) tailors the architecture to the netlist: it drops the unused BRAM columns entirely and repositions the DSPs, under an aspect ratio that lets VTR auto-size the same logic down to 36×40 . The dashed outline is the traditional footprint and the hatched strip is the reclaimed area — a 22% smaller fabric, the dominant term in the 15% ADP reduction. Gaps between tiles are routing channels.

Two reliability findings qualify these results. In-pool fitting is far more seed-sensitive than zero-shot transfer (aggregate 23.11–35.73% across seeds, driven by or1200 and boundtop), and deterministic zero-shot numbers diverge from stochastic sampling (negative for 4 of 6 benchmarks). A single-seed or single-rollout number is not a stable characterization (Section 5.6).

Three gaps are worth naming: results rest on three seeds and a systematic stochastic-rollout evaluation remains outstanding; the action space covers placement and aspect ratio only, where co-evolving CLB LUT size and arithmetic-block sizing [3] is a natural GCN extension; and broadening to the full GOLDS/NSGA-II benchmark set would allow direct comparisons without the metric conversion of Section 5.3.

References

- [1] Aman Arora, Andrew Boutros, Daniel Rauch, Aishwarya Rajen, Aatman Borda, Seyed A. Damghani, Samidh Mehta, Sangram Kate, Pragnesh Patel, Kenneth B. Kent, Vaughn Betz, and Lizy K. John. 2021. Koios: A Deep Learning Benchmark Suite for FPGA Architecture and CAD Research. In *31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE. doi:10.1109/FPL53798.2021.00010
- [2] Vaughn Betz and Jonathan Rose. 1997. VPR: A New Packing, Placement and Routing Tool for FPGA Research. In *7th International Workshop on Field-Programmable Logic and Applications (FPL)*. Springer-Verlag, 213–222. doi:10.1007/3-540-63465-7_226
- [3] D. V. Bhargav, G. Pradyumna, and Madhav Rao. 2025. Meta-Heuristic Optimization of Custom Heterogeneous Blocks Defined eFPGA Design. In *38th International Conference on VLSI Design (VLSID)*. IEEE, 326–331. doi:10.1109/VLSID64188.2025.00069
- [4] Ruichen Chen, Shengyao Lu, Mohamed A. Elgammal, Peter Chun, Vaughn Betz, and Di Niu. 2023. VPR-Gym: A Platform for Exploring AI Techniques in FPGA Placement Optimization. In *33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 72–78. doi:10.1109/FPL60245.2023.00018
- [5] Ruoyu Cheng and Junchi Yan. 2021. On Joint Learning for Solving Placement and Routing in Chip Design. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 16508–16519.
- [6] Luca Collini, Jitendra Bhandari, Chiara Muscari Tomajoli, Abdul Khader Thakkattu Moosa, Benjamin Tan, Xifan Tang, Pierre-Emmanuel Gaillardon, Ramesh Karri, and Christian Pilato. 2025. ARIANNA: An Automatic Design Flow for Fabric Customization and eFPGA Redaction. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* (2025). volume/issue/pages unverified at press time. doi:10.1145/3737287
- [7] Seyed Alireza Damghani and Kenneth B. Kent. 2022. Yosys+Odin-II: The Odin-II Partial Mapper with Yosys Coarse-grained Netlists in VTR. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. 157. doi:10.1145/3490422.3502344
- [8] Voktho Das, Kimia Zamiri Azar, and Hadi M. Kamali. 2026. NuRedact: Non-Uniform eFPGA Architecture for Low-Overhead and Secure IP Redaction. In *Design, Automation & Test in Europe Conference (DATE)*. IEEE.
- [9] Mohamed A. Elgammal, Amin Mohaghegh, Soheil G. Shahrour, Fatemehsadat Mahmoudi, Fabrice Kosar, Kimia Talei, Joshua Fife, Daniel Khadivi, Kevin E. Murray, Andrew Boutros, Kenneth B. Kent, Jeffrey B. Goeders, and Vaughn Betz. 2025. VTR 9: Open-Source CAD for Fabric and Beyond FPGA Architecture Exploration. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 18, 3 (2025), 39:1–39:53. doi:10.1145/3734798
- [10] Mohamed A. Elgammal, Kevin E. Murray, and Vaughn Betz. 2022. RLPlace: Using Reinforcement Learning and Smart Perturbations to Optimize FPGA Placement.

- IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 41, 8 (2022), 2532–2545. doi:10.1109/TCAD.2021.3109863
- [11] Anna Goldie, Azalia Mirhoseini, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nova, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2024. Addendum: A graph placement methodology for fast chip design. *Nature* 634, 8034 (2024), E10–E11. doi:10.1038/s41586-024-08032-5
 - [12] Yunbo Hou, Haoran Ye, Shuwen Yang, Yingxue Zhang, Siyuan Xu, and Guojie Song. 2025. TransPlace: Transferable Circuit Global Placement via Graph Neural Network. *arXiv preprint arXiv:2501.05667* (2025).
 - [13] Hao-Hsiang Hsiao, Yi-Chen Lu, Pruek Vanna-Iampikul, and Sung Kyu Lim. 2024. FastTuner: Transferable Physical Design Parameter Optimization using Fast Reinforcement Learning. In *International Symposium on Physical Design (ISPD)*. ACM.
 - [14] Shengyi Huang and Santiago Ontañón. 2022. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. In *Proceedings of the 35th International FLAIRS Conference*, Vol. 35. doi:10.32473/flairs.v35i.130584
 - [15] Hadi M. Kamali, Kimia Z. Azar, Farimah Farahmandi, and Mark Tehranipoor. 2023. SheLL: Shrinking eFPGA Fabrics for Logic Locking. In *Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–6.
 - [16] Yao Lai, Jinxin Liu, Zhentao Tang, Bin Wang, Jianye Hao, and Ping Luo. 2023. ChiPFormer: Transferable Chip Placement via Offline Decision Transformer. In *Proceedings of the 40th International Conference on Machine Learning (ICML) (PMLR, Vol. 202)*. 18346–18364.
 - [17] Yao Lai, Yao Mu, and Ping Luo. 2022. MaskPlace: Fast Chip Placement via Reinforced Visual Representation Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 24019–24030.
 - [18] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nova, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2021. A graph placement methodology for fast chip design. *Nature* 594, 7862 (2021), 207–212. doi:10.1038/s41586-021-03544-w
 - [19] Kevin E. Murray, Oleg Petelin, Sheng Zhong, Jia Min Wang, Mohamed Eldafrawy, Jean-Philippe Legault, Eugene Sha, Aaron G. Graham, Jean Wu, Matthew J. P. Walker, Hanqing Zeng, Panagiotis Patros, Jason Luu, Kenneth B. Kent, and Vaughn Betz. 2020. VTR 8: High-performance CAD and Customizable FPGA Architecture Modelling. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 13, 2 (2020), 9:1–9:55. doi:10.1145/3388617
 - [20] Pratyush Nandi, Anubhav Mishra, and Madhav Rao. 2024. GOLDS: Genetic Algorithm-based Optimization of Custom FPGA Architecture Layout Design for Secure Silicon. In *Proceedings of the Great Lakes Symposium on VLSI (GLSVLSI)*. ACM, 92–97. doi:10.1145/3649476.3658743
 - [21] Chiara Muscari Tomajoli, Luca Collini, Jitendra Bhandari, Abdul Khader Thakkattu Moosa, Benjamin Tan, Xifan Tang, Pierre-Emmanuel Gaillardon, Ramesh Karri, and Christian Pilato. 2022. ALICE: An Automatic Design Flow for eFPGA Redaction. In *59th ACM/IEEE Design Automation Conference (DAC)*. 781–786. doi:10.1145/3489517.3530543
 - [22] Nan Wu et al. 2023. GAMORA: Graph Learning based Symbolic Reasoning for Large-Scale Boolean Networks. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.